

[Vai al contenuto](#)

DISPENSA DELLA LEZIONE

Timing Functions

Eseguire codice in modo differito o ripetuto nel tempo con `setTimeout` e `setInterval`.

CAPITOLO 1

Cosa sono le timing functions

NOTA

Le timing functions permettono di eseguire una funzione in un momento preciso nel futuro, senza interrompere il flusso normale del programma.

In JavaScript è possibile programmare l'esecuzione di una funzione in un momento futuro. Le **timing functions** sono strumenti che permettono di dire al browser: "esegui questo codice dopo un certo numero di millisecondi". Questo è utile ogni volta che vuoi introdurre un ritardo, creare un'animazione, aggiornare un dato periodicamente o mostrare un messaggio dopo qualche secondo.

Il tempo si misura in **millisecondi**: 1000 millisecondi corrispondono a 1 secondo. Le due funzioni principali sono `setTimeout`, che esegue il codice una volta sola dopo un ritardo, e `setInterval`, che lo esegue ripetutamente a intervalli regolari.

Esempio

```
// 1000 millisecondi = 1 secondo  
// 500 millisecondi = mezzo secondo  
// 2000 millisecondi = 2 secondi
```

CAPITOLO 2

setTimeout

NOTA

`setTimeout` esegue una funzione **una volta sola** dopo il ritardo indicato.

`setTimeout` accetta due argomenti: la funzione da eseguire e il numero di millisecondi da aspettare prima di eseguirla. Dopo che il tempo è trascorso, la funzione viene chiamata una sola volta. Il codice che segue `setTimeout` non si ferma ad aspettare: continua a girare normalmente mentre il timer conta in background.

`setTimeout` restituisce un identificatore numerico, chiamato **timer ID**. Se vuoi annullare il timer prima che scatti, puoi passare quell'ID a `clearTimeout`. In questo modo la funzione non verrà mai eseguita.

Sintassi

```
setTimeout(funzione, millisecondi);
```

Esempio

```
// Stampa un messaggio dopo 2 secondi
setTimeout(() => {
  console.log('Sono passati 2 secondi!');
}, 2000);

// Il codice qui sotto viene eseguito subito, senza aspettare
console.log('Questo appare prima, anche se è scritto dopo.');
```

Esempio con clearTimeout

```
// Avvia un timer
const timerID = setTimeout(() => {
  console.log('Questo non verrà mai stampato.');
```



```
// Annulla il timer prima che scatti
clearTimeout(timerID);
```

CAPITOLO 3

setInterval

NOTA

`setInterval` esegue una funzione **ripetutamente** ogni tot millisecondi, finché non viene fermato con `clearInterval`.

`setInterval` funziona in modo simile a `setTimeout`, ma invece di eseguire la funzione una volta sola, la richiama automaticamente ogni volta che il tempo indicato è trascorso. Questo lo rende utile per operazioni che devono ripetersi nel tempo: aggiornare un contatore, mostrare l'ora corrente, fare lampeggiare un elemento o controllare una condizione a intervalli regolari.

Come `setTimeout`, anche `setInterval` restituisce un timer ID. Per fermare il ciclo si usa `clearInterval` passando quell'ID come argomento. Senza `clearInterval`, la funzione continua a girare all'infinito finché la pagina è aperta.

Sintassi

```
setInterval(funzione, millisecondi);
```

Esempio

```
// Stampa un messaggio ogni secondo
const intervallo = setInterval(() => {
  console.log('Tick!');
}, 1000);
```

Esempio con clearInterval

```
// Conta da 1 a 5, poi si ferma  
let contatore = 0;  
  
const intervallo = setInterval(() => {  
  contatore = contatore + 1;  
  console.log('Contatore: ' + contatore);  
  
  if (contatore === 5) {  
    clearInterval(intervallo);  
    console.log('Contatore fermato.');  
  }  
}, 1000);
```